

Estrutura do Projeto FinView AI

Documento gerado para explicar a organização técnica do projeto, os principais arquivos, os fluxos da aplicação e os pontos de manutenção.

1. Visão Geral

O FinView AI é uma aplicação web desenvolvida em Django para controle financeiro pessoal. O sistema permite que o usuário crie uma conta, cadastre sua renda mensal, registre despesas, visualize indicadores financeiros em um dashboard e receba análises geradas por IA com base nos dados cadastrados.

A aplicação é composta por:

- Backend Django, responsável por autenticação, regras de negócio, persistência e rotas.
- Templates HTML em `templates/gastos`, responsáveis pelas telas.
- Assets estáticos em `static/assets`, responsáveis pelo CSS e JavaScript.
- Integração com Groq AI em `gastos/ia.py`.
- Deploy preparado para Render com Gunicorn, WhiteNoise e PostgreSQL via variável `DATABASE_URL`.

1.1. Ferramenta Principal de Apoio ao Desenvolvimento

Durante a organização, documentação e evolução técnica deste projeto, foi utilizado o Codex como ferramenta principal de apoio ao desenvolvimento.

O Codex atuou como agente de engenharia para:

- Inspecionar a estrutura real do repositório.
- Ajustar arquivos do projeto Django.
- Criar documentação técnica.
- Gerar artefatos de apresentação.
- Apoiar correções de deploy, banco de dados, integração com IA e frontend.
- Executar validações locais, como `python3 manage.py check` e `python3 manage.py test gastos`.

Modelo de linguagem utilizado:

Codex, agente baseado em GPT-5

O uso do Codex não substitui a aplicação em si. Ele foi usado como ferramenta de desenvolvimento, análise e documentação. A aplicação em produção continua sendo executada por Django, Gunicorn, PostgreSQL, WhiteNoise e integração com Groq AI.

1.2. Principais Prompts Utilizados no Projeto

Os prompts abaixo representam os principais comandos e pedidos usados durante a evolução do projeto. Eles foram organizados por objetivo para mostrar como o Codex foi acionado como ferramenta principal de apoio ao desenvolvimento.

Deploy e preparação para produção

Tabela: Prompt utilizado - Objetivo no projeto - Resultado esperado

"deixa o código pronto pra deploy no render pf" - Preparar a aplicação Django para rodar no Render. - Configuração de deploy com Gunicorn, WhiteNoise, variáveis de ambiente e banco PostgreSQL.

"já deixa tudo configurado" - Transformar a orientação de deploy em ajustes reais no repositório. - Arquivos como `build.sh`, `render.yaml`, dependências e configurações de produção ajustados.

"css não subiu ainda" - Corrigir problema visual em produção. - Diagnóstico do pipeline de static files e ajuste para servir CSS corretamente no Render.

Integração com IA

Tabela: Prompt utilizado - Objetivo no projeto - Resultado esperado

"me ajuda a configura a integração com a groq AI, API_KEY tá salvo no arquivo .env" - Ativar a integração com Groq AI usando variável de ambiente. - Leitura segura da chave, cliente Groq configurado e dependência adicionada.

"o resultado da análise da IA, está no painel dica? Se sim, informa que análise foi gerada por IA" - Deixar claro para o usuário que a recomendação vem de IA. - Painel identificado como "Análise gerada por IA".

"a análise do gasto está alinhado com o objetivo? se não, vamos fazer essa amarração" - Conectar o insight da IA ao objetivo financeiro escolhido no cadastro. - Prompt/contexto da IA usando renda, despesas, saldo e objetivo financeiro do usuário.

"no site, vamos implementar uma relação de metas do usuário" - Evoluir a análise para considerar metas concretas, como guardar R\$ 6.000,00. - Prompt/contexto da IA usando renda, gastos, objetivo financeiro e metas cadastradas.

Construção do prompt de análise dos gastos

A análise financeira da IA foi construída em três camadas: instrução fixa do sistema, contexto financeiro gerado pela aplicação e pergunta enviada pelo usuário/interface.

1. Prompt de sistema

Local: `gastos/ia.py`.

Esse prompt define o papel da IA dentro do FinView AI:

```
Voce e o assistente financeiro do FinView AI.
Responda em portugues do Brasil, com tom direto e util.
Use somente os dados financeiros enviados pelo sistema.
Sempre relacione a analise ao objetivo financeiro do usuario quando ele for informado.
Responda em topicos curtos, sem texto corrido longo.
Se os dados forem insuficientes, diga o que falta cadastrar.
Evite prometer resultados e nao trate isso como consultoria financeira profissional.
```

Objetivo dessa camada:

- Limitar a resposta aos dados cadastrados no sistema.
- Evitar respostas longas ou genéricas.
- Manter tom direto, útil e em português do Brasil.
- Amarrar a análise ao objetivo financeiro do usuário.

- Deixar claro quando faltam dados, como renda ou despesas.
- Evitar promessa de resultado financeiro.

2. Contexto financeiro enviado para a IA

Local: `gastos/views.py`, função `ai_context_for_month`.

A aplicação monta um resumo com os principais dados do usuário:

```

Usuario: nome do usuario
Objetivo financeiro: objetivo cadastrado no perfil
Mes de referencia: mes analisado
Renda cadastrada: total de rendas do mes
Total de despesas: total de gastos do mes
Saldo: renda menos despesas
Percentual comprometido: quanto da renda foi consumida
Categorias: total por categoria de gasto
Prioridades: total por prioridade
Metas financeiras: alvo, valor guardado, valor restante, progresso e valor necessario por mes

```

Objetivo dessa camada:

- Dar dados reais para a IA analisar.
- Impedir que a IA dependa de suposições.
- Conectar gastos, renda, objetivo financeiro e metas.
- Permitir que a IA diga se a meta é viável no ritmo atual.

3. Prompt da interface para análise dos gastos

Locais:

- `static/assets/js/app.js`
- `gastos/views.py`

Prompt atual:

```

Analise se os gastos e receitas do mes estao alinhados ao objetivo financeiro e as metas cadastradas do usu

```

Objetivo dessa camada:

- Pedir uma análise prática do mês.
- Considerar receitas e despesas juntas, não apenas gastos isolados.
- Relacionar a resposta ao objetivo financeiro e às metas cadastradas.
- Forçar uma saída curta, escaneável e fácil de exibir no painel lateral.
- Padronizar a resposta em cinco blocos: situação, meta, viabilidade, ponto de atenção e ação prática.

Evolução do prompt

O prompt começou com uma análise simples de gastos:

```

Analise se os gastos do mes estao alinhados ao objetivo financeiro do usuario.

```

Depois evoluiu para incluir renda, saldo e metas:

Analise se os gastos e receitas do mes estao alinhados ao objetivo financeiro e as metas cadastradas do usu

Essa mudança foi importante porque a IA deixou de apenas comentar despesas e passou a atuar como apoio de planejamento financeiro. Com metas cadastradas, ela consegue responder se o usuário está no caminho para atingir um alvo concreto, como guardar R\$ 6.000,00.

Interface e experiência do usuário

Tabela: Prompt utilizado - Objetivo no projeto - Resultado esperado

"Na aba de criação de conta e seleção de objetivo, avise que essa informação será importante para análise da IA" - Melhorar a clareza do cadastro. - Texto informativo explicando que o objetivo financeiro orienta a análise da IA.

"Na visualização da informação, o texto tá mal distribuído, a melhor coisa seria ordenar em tópicos" - Melhorar a legibilidade do retorno da IA. - Renderização da análise em tópicos/listas em vez de parágrafo único.

"quando digitamos valores, a separação de casas decimais não está acontecendo" - Melhorar o preenchimento de valores monetários. - Máscara para transformar entradas como 1500 em 1.500,00.

Prompt de front-end / protótipo visual

O prompt principal usado para orientar o front-end pediu a criação de uma aplicação web chamada FinView AI, com foco em organização financeira pessoal apoiada por IA.

Resumo do prompt:

- Criar um front-end completo, navegável e moderno para uma fintech inteligente.
- Usar uma identidade visual premium, tecnológica e responsiva, com fundo escuro, gradientes suaves, glassmorphism, blur, cards translúcidos, brilho sutil e elementos 3D leves.
- Considerar uma stack visual baseada em React, Vite, Tailwind CSS, React Router, Three.js, React Three Fiber, Drei, Motion/Framer Motion e gráficos com Recharts ou dados simulados.
- Criar páginas para landing page, login, cadastro, entrada mensal, cadastro de gastos, dashboard financeiro com painel lateral de IA e perfil do usuário.
- No dashboard, destacar cards de renda, gastos, saldo e score financeiro, além de gráficos por categoria, prioridade, recorrência e evolução mensal.
- Incluir um painel lateral de IA colapsável, com análise automática, alertas, sugestões de economia, recomendações baseadas no objetivo financeiro e campo de chat.
- Garantir boa experiência de uso com navegação clara, microinterações, cards animados, botões com hover elegante e interface adaptada para desktop e mobile.
- Usar dados fictícios para simular gráficos, despesas e recomendações, sem necessidade de backend ou autenticação real nessa etapa de prototipação visual.

Resultado esperado do prompt:

Uma aplicação visualmente sofisticada, funcional como protótipo navegável e com aparência de fintech moderna, deixando clara a proposta do FinView AI: transformar dados financeiros em decisões melhores com apoio de IA.

Correções de erro e estabilidade

Tabela: Prompt utilizado - Objetivo no projeto - Resultado esperado

"o git bloqueou meu commit e agora estou com erro" - Resolver bloqueio por segredo versionado. - Remoção do .env do histórico local, ajuste do .gitignore e push limpo.

"Eu tenho um valor de mil reais cadastrados, se eu colocar mais 500 reais como renda variável, ele da erro 500" - Corrigir erro de produção ao cadastrar renda adicional. - Migração e fallback para permitir múltiplas entradas de renda no mesmo mês.

"testa o python manage.py runserve, tem dado erro aqui" - Validar execução local do Django. - Identificação do comando correto runserver e verificação local do servidor.

Documentação e apresentação

Tabela: Prompt utilizado - Objetivo no projeto - Resultado esperado

"precisamos explicar toda a estrutura do projeto (gere um pdf para isso)" - Criar documentação técnica do projeto. - Documento .md e PDF explicando arquitetura, arquivos, fluxos e manutenção.

"faça um prompt para desenhar um fluxograma da aplicação (fluxo do sistema e fluxo do user)" - Estruturar o desenho dos fluxos. - Prompt orientado para gerar fluxograma do usuário e do sistema.

"pode gerar o fluxograma? com a saída em pdf? se sim faça" - Transformar o fluxo em artefato visual. - Fluxograma exportável em PDF.

"consegue gerar uma apresentação tbm?" - Preparar material de apresentação do projeto. - Arquivo .pptx com visão geral, estrutura, fluxos, IA, deploy e manutenção.

"foi citado a utilização do codex como ferramenta principal? (informar o modelo de linguagem tbm)" - Registrar a ferramenta de apoio usada no desenvolvimento. - Slide e seção informando uso do Codex e modelo baseado em GPT-5.

2. Estrutura de Diretórios

```
FinView_Ai/
├── manage.py
├── requirements.txt
├── build.sh
├── Procfile
├── render.yaml
├── .env.example
├── .gitignore
├── db.sqlite3
├── config/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── gastos/
│   ├── admin.py
│   ├── apps.py
│   ├── ia.py
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   ├── views.py
│   └── migrations/
├── templates/
│   └── gastos/
├── static/
│   ├── assets/
│   │   ├── css/
│   │   └── js/
├── docs/
└── prompts/
```

3. Arquivos de Raiz

`manage.py`

Arquivo padrão do Django usado para executar comandos administrativos:

- `python manage.py runserver`
- `python manage.py migrate`
- `python manage.py test`
- `python manage.py collectstatic`

`requirements.txt`

Lista as dependências Python da aplicação:

- Django: framework web.
- `django-environ`: leitura de variáveis do `.env`.
- `gunicorn`: servidor WSGI usado em produção.
- `psycopg[binary]`: conexão com PostgreSQL.
- `whitenoise`: entrega de arquivos estáticos em produção.
- `groq`: SDK usado para consultar a IA.

`build.sh`

Script executado no build do Render:

```
pip install -r requirements.txt
python manage.py collectstatic --no-input
python manage.py migrate --no-input
```

Ele instala dependências, coleta arquivos estáticos e aplica migrações no banco de produção.

`Procfile`

Define o comando de inicialização compatível com plataformas como Render/Heroku:

```
web: gunicorn config.wsgi:application --bind 0.0.0.0:$PORT
```

`render.yaml`

Blueprint de deploy no Render. Define:

- Serviço web Python chamado `finview-ai`.
- Comando de build: `bash build.sh`.
- Comando de start: Gunicorn com `config.wsgi`.
- Variáveis de ambiente necessárias, como `SECRET_KEY`, `DATABASE_URL`, `GROQ_API_KEY`, `ALLOWED_HOSTS` e `CSRF_TRUSTED_ORIGINS`.

`env.example`

Modelo de variáveis de ambiente. Não contém valores reais de segredo, apenas exemplos de chaves esperadas.

``.gitignore``

Evita versionar arquivos locais/sensíveis, principalmente:

- `.env`
- `.venv/`
- `__pycache__/`
- `*.py[cod]`
- `staticfiles/`
- `.DS_Store`

``.db.sqlite3``

Banco SQLite local usado em desenvolvimento quando `DATABASE_URL` não aponta para outro banco. Em produção, o projeto usa PostgreSQL no Render.

4. Aplicação Django Principal: ``.config/``

``.config/settings.py``

Centraliza as configurações do Django:

- Lê `.env` usando `django-environ`.
- Define `INSTALLED_APPS`, incluindo o app `gastos`.
- Configura banco de dados via `DATABASE_URL`.
- Configura templates em `templates/`.
- Configura arquivos estáticos com `WhiteNoise`.
- Configura segurança de cookies, SSL e HSTS.
- Define as variáveis da Groq:
 - `GROQ_API_KEY`
 - `GROQ_MODEL`

O banco padrão é SQLite local, mas em produção o Render deve fornecer `DATABASE_URL` com PostgreSQL.

``.config/urls.py``

Roteia:

- `/admin/` para o Django Admin.
- `/` para as URLs do app `gastos`.

`config/wsgi.py` e `config/asgi.py`

Entrypoints de servidor. O deploy usa `config.wsgi:application` com Gunicorn.

5. App Principal: `gastos/`

O app `gastos` concentra os modelos financeiros, views, rotas, integração com IA e testes.

`gastos/models.py`

Define os modelos de dados:

`UserProfile`

Extensão do usuário padrão do Django.

Campos:

- `user`: relacionamento um-para-um com `auth.User`.
- `goal`: objetivo financeiro principal do usuário.

Um signal `post_save` cria automaticamente o perfil quando um novo usuário é criado.

`MonthlyIncome`

Representa entradas de renda mensal.

Campos:

- `user`
- `amount`
- `income_type`: `fixed` ou `variable`
- `reference_month`
- `created_at`
- `updated_at`

O sistema permite múltiplas rendas no mesmo mês. Caso algum banco antigo ainda mantenha uma restrição única, a view tem fallback para somar o novo valor à renda existente, evitando erro 500.

`Expense`

Representa despesas cadastradas pelo usuário.

Campos:

- user
- name
- amount
- date
- category
- recurrence: fixed ou variable
- priority: essential, important ou superfluous
- notes
- created_at
- updated_at

Também possui a propriedade `amount_display`, que formata valores em reais.

FinancialGoal

Representa as metas financeiras mensuráveis do usuário.

Exemplo de uso:

- Guardar R\$ 6.000,00 para uma reserva de emergência.
- Quitar uma dívida.
- Juntar dinheiro para uma compra.
- Criar uma meta de investimento.

Campos:

- user
- name: nome da meta.
- target_amount: valor alvo.
- saved_amount: valor já guardado.
- target_date: prazo desejado.
- goal_type: tipo da meta, como economia, dívida, investimento, compra ou reserva.
- priority: prioridade da meta.
- status: ativa, concluída ou pausada.
- notes
- created_at
- updated_at

Propriedades calculadas:

- remaining_amount: quanto ainda falta para atingir a meta.
- progress_percent: percentual concluído.
- months_remaining: meses restantes até o prazo.

- `required_monthly_amount`: valor que precisa ser guardado por mês.
- `is_completed`: indica se a meta já foi concluída.

Essa estrutura transforma o objetivo financeiro geral em algo mensurável. O objetivo do perfil continua sendo uma intenção ampla, como "Economizar dinheiro"; a meta financeira vira um alvo concreto, como "Guardar R\$ 6.000,00 até dezembro".

`gastos/urls.py`

Rotas principais:

- `/`: home.
- `/landing/`: página pública de apresentação.
- `/login/`: login.
- `/signup/`: criação de conta.
- `/logout/`: saída.
- `/dashboard/`: dashboard financeiro.
- `/goals/`: cadastro e acompanhamento de metas financeiras.
- `/goals/<id>/delete/`: exclusão de meta financeira.
- `/monthly/`: cadastro e visualização mensal de renda/despesas.
- `/monthly/delete-income/`: exclusão das rendas do mês.
- `/new-expense/`: criação de despesa.
- `/expenses/<id>/edit/`: edição de despesa.
- `/expenses/<id>/delete/`: exclusão de despesa.
- `/profile/`: perfil do usuário.
- `/ai/insight/`: endpoint usado pelo painel da IA.

`gastos/views.py`

Contém as regras de negócio e renderização de telas.

Principais responsabilidades:

- Autenticação e cadastro de usuários.
- Conversão de datas e valores monetários.
- Cálculo de renda, despesas, saldo, percentual comprometido e score.
- Geração de resumos por categoria, prioridade e recorrência.
- Cadastro, listagem e exclusão de metas financeiras.
- Cálculo de progresso, valor restante e valor mensal necessário para cumprir uma meta.
- Criação automática de despesas fixas em meses futuros.
- Cadastro, edição e exclusão de despesas.
- Cadastro e soma de rendas mensais.
- Montagem do contexto enviado para a IA.

- Endpoint JSON para insights da Groq.

Helpers importantes

- `money(value)`: formata Decimal para R\$.
- `decimal_from_post(value)`: converte valores enviados por formulário para Decimal.
- `month_from_input(value)`: converte YYYY-MM em data do primeiro dia do mês.
- `ensure_fixed_expenses_for_month(user, reference_month)`: replica despesas fixas de meses anteriores.
- `ai_context_for_month(user, reference_month)`: monta o contexto financeiro usado pela IA.
- `add_monthly_income(...)`: cria uma renda mensal ou soma ao registro existente se houver bloqueio de banco legado.
- `format_goal(goal, monthly_balance)`: prepara os dados da meta para exibição no dashboard e na tela de metas.
- `goals_context_for_user(user, monthly_balance)`: monta o contexto de metas ativas do usuário.

`gastos/ia.py`

Responsável pela integração com Groq AI.

Principais elementos:

- `SYSTEM_PROMPT`: define o comportamento esperado da IA.
- `groq_configured()`: verifica se existe chave configurada.
- `groq_client()`: instancia o cliente da Groq.
- `gerar_resposta_financeira(...)`: envia o contexto e a pergunta para a IA.

A IA recebe:

- Objetivo financeiro do usuário.
- Metas financeiras ativas.
- Valor alvo, valor já guardado, valor restante e ritmo mensal necessário.
- Mês de referência.
- Renda cadastrada.
- Total de despesas.
- Saldo.
- Percentual comprometido.
- Categorias e prioridades.

A resposta é solicitada em tópicos curtos para facilitar a leitura no painel lateral.

Com as metas financeiras, a IA passa a responder de forma mais estratégica. Ela não analisa apenas se os gastos estão bons ou ruins; ela também verifica se o saldo mensal e a composição das despesas ajudam o usuário a chegar na meta cadastrada.

Exemplo:

Meta: guardar R\$ 6.000,00.
Valor já guardado: R\$ 500,00.
Valor restante: R\$ 5.500,00.
Prazo: 6 meses.
Ritmo necessário: R\$ 916,67 por mês.

Com esses dados, a IA pode sugerir redução de gastos variáveis, revisão de despesas supérfluas, aumento de prazo ou aumento de renda.

`gastos/admin.py`

Registra modelos no Django Admin para inspeção e manutenção manual.

`gastos/tests.py`

Testes automatizados cobrindo:

- Integração configurável com Groq.
- Resposta adequada quando a chave da IA não existe.
- Inclusão do objetivo financeiro no contexto da IA.
- Inclusão de metas financeiras no contexto da IA.
- Criação e renderização da página de metas.
- Exibição de meta financeira no dashboard.
- Cadastro de múltiplas rendas no mesmo mês.
- Fallback quando o banco ainda possui restrição única antiga.

6. Templates: `templates/gastos/`

Os templates definem as telas da aplicação.

Templates base

- `base.html`: estrutura HTML base, CSS global e blocos de conteúdo.
- `app.html`: layout interno autenticado com sidebar, navegação mobile e painel da IA.
- `auth.html`: base para telas de autenticação.

Telas públicas

- `landing.html`: página de apresentação do produto.
- `home.html` e `index.html`: telas auxiliares/entrada.

Autenticação

- `login.html`: formulário de login.
- `signup.html`: criação de conta, com escolha de objetivo financeiro. Esse objetivo é usado nas análises da IA.

Área autenticada

- `dashboard.html`: indicadores financeiros, gráficos/resumos, histórico e top despesas.
- `goals.html`: cadastro e acompanhamento das metas financeiras do usuário.
- `monthly.html`: cadastro de renda mensal, resumo do mês e lista de despesas.
- `new-expense.html`: formulário de criação e edição de despesa.
- `profile.html`: informações do usuário e preferências.

Partials

Os partials ficam em `templates/gastos/partial/` e evitam duplicação:

- `ai_panel.html`: botão e painel lateral da IA.
- `sidebar.html`: menu lateral.
- `mobile_nav.html`: navegação mobile.
- `background.html`: elementos visuais de fundo.
- `logo.html`: marca do projeto.

7. Assets Estáticos: ``static/assets/``

``static/assets/css/styles.css``

Define o visual do projeto:

- Tema escuro.
- Cards com efeito glass.
- Layout responsivo.
- Sidebar e mobile nav.
- Dashboard, tabelas e barras.
- Cards e estados visuais de metas financeiras.
- Painel lateral da IA.
- Estados visuais de alerta, dica, objetivo e sucesso.

``static/assets/js/app.js``

Responsável por comportamentos de frontend:

- Abrir e fechar o painel da IA.
- Buscar insight em `/ai/insight/`.
- Renderizar resposta da IA em tópicos.
- Enviar para a IA um prompt que considera objetivo financeiro e metas cadastradas.
- Normalizar e formatar campos de dinheiro.

- Limpar e-mails, nomes e textos ao sair do campo.

Para valores monetários:

- Digitação visual: 1500 vira 1.500,00.
- Envio ao backend: 1.500,00 vira 1500.00.

8. Fluxos Principais

Cadastro de usuário

1. Usuário acessa /signup/.
2. Informa nome, e-mail, senha e objetivo financeiro.
3. Django cria `auth.User`.
4. Signal cria `UserProfile`.
5. View salva o objetivo financeiro escolhido.
6. Usuário é autenticado e redirecionado para /monthly/.

Login

1. Usuário acessa /login/.
2. Informa e-mail e senha.
3. Django autentica usando o e-mail como username.
4. Se válido, redireciona para /dashboard/.

Cadastro de renda mensal

1. Usuário acessa /monthly/.
2. Informa renda, tipo e mês.
3. Frontend formata o valor em reais.
4. No submit, JS converte para formato numérico.
5. Backend usa `decimal_from_post`.
6. `add_monthly_income` cria a entrada.
7. Se houver restrição antiga no banco, soma ao registro existente para evitar erro 500.

Cadastro de despesa

1. Usuário acessa /new-expense/.
2. Informa nome, valor, data, categoria, recorrência, prioridade e notas.
3. Backend cria `Expense`.
4. Usuário retorna para a visão mensal do mês correspondente.

Dashboard

1. Usuário acessa `/dashboard/`.
2. Backend filtra renda e despesas do mês.
3. Calcula total de renda, total de gastos, saldo e percentual comprometido.
4. Gera score financeiro.
5. Agrupa gastos por categoria, prioridade e recorrência.
6. Renderiza indicadores e listas.

Despesas fixas

Quando um mês é acessado, `ensure_fixed_expenses_for_month` verifica despesas fixas anteriores e cria cópias no mês atual se ainda não existirem. Isso ajuda a projetar gastos recorrentes.

Insight da IA

1. Usuário abre o painel lateral.
2. JavaScript faz POST para `/ai/insight/`.
3. View valida autenticação e chave Groq.
4. Backend monta contexto financeiro.
5. `gastos/ia.py` envia prompt para Groq.
6. Resposta volta em JSON.
7. Frontend renderiza o texto em tópicos.

9. Banco de Dados

Desenvolvimento

Por padrão, se `DATABASE_URL` não estiver definido, o Django usa:

```
db.sqlite3
```

Esse arquivo é útil para desenvolvimento local, mas não deve ser tratado como fonte principal de produção.

Produção

No Render, o banco deve ser PostgreSQL via:

```
DATABASE_URL
```

O `build.sh` aplica as migrações automaticamente no deploy.

Migrações importantes

- `0001_initial`: cria modelos principais.

- 0002_monthlyincome_multiple_entries: ajusta renda mensal para aceitar tipos atuais.
- 0003_remove_monthlyincome_unique_db_constraint: remove restrição antiga de renda mensal.
- 0004_drop_monthlyincome_unique_constraint: reforça a remoção da restrição única em bancos que ainda tenham esse bloqueio.

10. Integração com IA

A integração usa a API da Groq.

Variáveis necessárias:

```
GROQ_API_KEY
GROQ_MODEL
```

Também existe fallback local para API_KEY, mas em produção a variável recomendada é GROQ_API_KEY.

O objetivo financeiro escolhido no cadastro é enviado para a IA e influencia a análise. Exemplo: se o objetivo for "Economizar dinheiro", a IA avalia se os gastos do mês estão alinhados a esse objetivo.

10.1. Metas Financeiras com IA

A funcionalidade de metas permite que o usuário cadastre um alvo financeiro concreto e acompanhe se o comportamento mensal está ajudando ou atrapalhando esse plano.

Exemplo:

- Meta: guardar R\$ 6.000,00.
- Valor já guardado: R\$ 500,00.
- Valor restante: R\$ 5.500,00.
- Prazo: 6 meses.
- Ritmo necessário: aproximadamente R\$ 916,67 por mês.

A aplicação cruza essa meta com:

- Renda mensal cadastrada.
- Total de gastos do mês.
- Saldo restante.
- Categorias de despesas.
- Prioridades das despesas.
- Objetivo financeiro geral do usuário.

Com isso, a IA pode responder perguntas como:

- A meta é viável no prazo informado?
- Quanto o usuário precisa guardar por mês?
- Quais categorias precisam ser reduzidas?

- O saldo atual é suficiente para manter o ritmo necessário?
- O prazo deveria ser ajustado?

Fluxo da funcionalidade:

1. Usuário acessa a página Metas.
2. Cadastra nome, valor alvo, valor já guardado, prazo, tipo e prioridade.
3. O sistema calcula progresso, valor restante e valor mensal necessário.
4. O dashboard exibe a meta principal.
5. O contexto enviado para a Groq inclui as metas ativas.
6. A IA gera uma análise considerando receitas, gastos, objetivo financeiro e meta.

Arquivos principais:

- `gastos/models.py`: model `FinancialGoal`.
- `gastos/views.py`: `views goals`, `delete_goal`, helpers de cálculo e contexto para IA.
- `gastos/urls.py`: rotas `/goals/` e `/goals/<id>/delete/`.
- `templates/gastos/goals.html`: página de cadastro e acompanhamento.
- `templates/gastos/dashboard.html`: card de meta no dashboard.
- `static/assets/js/app.js`: prompt da IA atualizado para considerar metas.
- `gastos/migrations/0005_financialgoal.py`: migration da tabela de metas.
- `gastos/tests.py`: testes da criação, exibição e contexto de metas.

11. Deploy no Render

O deploy está preparado por:

- `render.yaml`
- `build.sh`
- `Procfile`
- `requirements.txt`
- `config/settings.py`

Fluxo esperado:

1. Render baixa o código do GitHub.
2. Executa `bash build.sh`.
3. Instala dependências.
4. Coleta estáticos.
5. Aplica migrações.
6. Inicia Gunicorn.

Variáveis obrigatórias no Render:

- SECRET_KEY
- DEBUG=False
- ALLOWED_HOSTS
- CSRF_TRUSTED_ORIGINS
- DATABASE_URL
- GROQ_API_KEY
- GROQ_MODEL
- SECURE_SSL_REDIRECT
- SECURE_HSTS_SECONDS

12. Segurança e Cuidados

Não versionar segredos

O `.env` não deve ir para o Git. Ele contém segredos como:

- SECRET_KEY
- URL de banco.
- Chave da Groq.

Rotacionar API keys expostas

Se uma chave aparecer em commit, print, chat ou log, o ideal é revogar e gerar uma nova.

Produção não usa SQLite

Mesmo que `db.sqlite3` exista localmente, produção deve usar PostgreSQL.

Logs de erro

Se produção mostrar erro 500, os pontos mais prováveis são:

- Migração não aplicada.
- Variável de ambiente ausente.
- Constraint antiga no banco.
- Erro de parse de valor monetário.
- Chave de IA ausente ou inválida.

13. Comandos Úteis

Rodar localmente

```
python3 manage.py runserver
```

Se a porta 8000 estiver ocupada:

```
python3 manage.py runserver 127.0.0.1:8001
```

Verificar configuração

```
python3 manage.py check
```

Rodar testes

```
python3 manage.py test gastos
```

Aplicar migrações

```
python3 manage.py migrate
```

Coletar arquivos estáticos

```
python3 manage.py collectstatic --no-input
```

Ver migrações pendentes

```
python3 manage.py migrate --plan
```

14. Pontos de Manutenção

Melhorias futuras possíveis

- Migrar templates duplicados para uma base única com extends.
- Criar models ou tabelas para categorias configuráveis.
- Criar histórico detalhado das análises da IA.
- Criar histórico de evolução das metas mês a mês.
- Permitir edição de metas financeiras já cadastradas.
- Exibir simulações de prazo para metas, comparando cenário atual e cenário recomendado.
- Adicionar testes de interface com Playwright.
- Melhorar logs de erro em produção.
- Adicionar página de configuração de perfil funcional.
- Separar serviços de domínio em módulos próprios para reduzir o tamanho de `views.py`.

Arquivos que concentram maior regra de negócio

- `gastos/views.py`
- `gastos/models.py`
- `gastos/ia.py`

- `static/assets/js/app.js`
- `templates/gastos/goals.html`

Esses são os principais pontos para entender antes de alterar comportamento financeiro ou integração com IA.

15. Resumo Executivo

O FinView AI é uma aplicação Django de controle financeiro pessoal. A estrutura é relativamente simples: um projeto Django (`config`) e um app central (`gastos`). O app cuida de autenticação, renda, despesas, metas financeiras, dashboard e IA. O deploy está preparado para Render, usando PostgreSQL em produção e SQLite localmente.

Com a funcionalidade de metas, o sistema passa a conectar análise mensal com planejamento financeiro. O usuário pode cadastrar um alvo como "guardar R\$ 6.000,00", e a aplicação calcula progresso, valor restante e ritmo mensal necessário. A IA usa essas informações junto com renda, gastos e objetivo financeiro para sugerir ações mais direcionadas.

O sistema depende fortemente de variáveis de ambiente e do fluxo de migrações. Para manter o projeto estável, os cuidados mais importantes são:

- Nunca versionar `.env`.
- Garantir `DATABASE_URL` e `GROQ_API_KEY` no Render.
- Rodar testes após mudanças em renda, despesas, metas ou IA.
- Verificar se o deploy aplicou migrações.
- Manter os dados de produção fora do `db.sqlite3` local.